U·tad

Unidad 1 — Introducción a JavaScript y al cliente web

Web Development I, the Client · 3.º de Ingeniería del Software · 2026–2027

Navegador y DevTools · ECMAScript 2025/2026 · DOM y eventos · Cuaderno de ejercicios 1 · Misión M1 «El Despertar del DOM»

Al terminar esta unidad sabrás…

- Explicar **qué hace el navegador** con tu HTML, CSS y JS (y por qué tocar el DOM cuesta).
- Usar las **DevTools** para inspeccionar, depurar con *breakpoints* y ver la red.
- Escribir JavaScript **moderno**: `const` / `let`, `===`, *template literals*, `??`, `?.`, *arrow functions*.
- Entender **tipos, coerción y conversión** (y por qué `"5" + 1` es `"51"`).
- Manipular el **DOM** con `querySelector`, `textContent`, `classList` y responder a **eventos**.
- Aplicar la **delegación de eventos** y evitar el XSS con `innerHTML`.
- Construir un juego completo en JS vanilla: **Caza al bug**.

Mapa de la unidad

1. ¿Qué es JavaScript?
2. El navegador y el desarrollo cliente
3. Cómo incluir JavaScript
4. Variables, tipos y coerción
5. Operadores y *strings*
6. Control de flujo y funciones
7. El DOM: seleccionar, modificar, eventos, **delegación**
8. Primera mirada a la asincronía
9. Buenas prácticas
10. **Reto jefe:** Caza al bug
11. **Arena:** misión M1 y defensa oral

> Ejercicios del cuaderno 1 intercalados: 4 calentamientos 🔴, 3 misiones 🟡, 1 reto jefe 🟢.

1 · ¿Qué es JavaScript?

El tercer pilar de la web: HTML **estructura**, CSS **presenta**, JavaScript **se comporta**.

Las tres capas del desarrollo cliente

Capa	Tecnología	Responsabilidad
Estructura	**HTML**	El contenido y su significado (semántica)
Presentación	**CSS**	El aspecto visual (colores, maquetación)
Comportamiento	**JavaScript**	La interactividad y la lógica

- Separarlas permite cambiar una sin tocar las otras y habilita la **mejora progresiva**: la página sirve con HTML/CSS y JS **añade** capacidades.
- **HTML semántico** (`<button>`, `<label>`, `<nav>`…) en lugar de `<div>` para todo: un `<button>` ya es accesible por teclado; un `<div>` clicable, no.

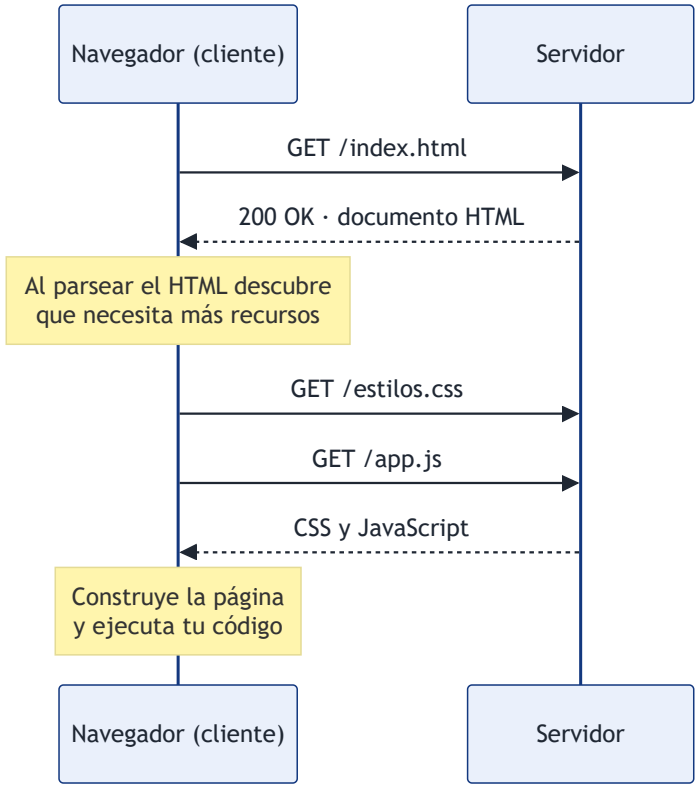
Un poco de historia (y de nombres)

- 1995: Brendan Eich lo crea en Netscape en **diez días**.
- **No tiene relación con Java**: fue marketing de la época.
- El estándar se llama **ECMAScript** (ECMA-262). Desde 2015 (ES6) sale **una versión al año**: hoy, ES2025/2026.
- «ES6», «ES2020», «ES2024»… son versiones del estándar; JavaScript es su implementación.
- Necesita un **motor**: V8 (Chrome, Node.js), SpiderMonkey (Firefox), JavaScriptCore (Safari).
- Con **Node.js** también corre en servidores y herramientas. Aquí nos centramos en el **cliente**: el navegador.

2 · El navegador y el desarrollo web cliente

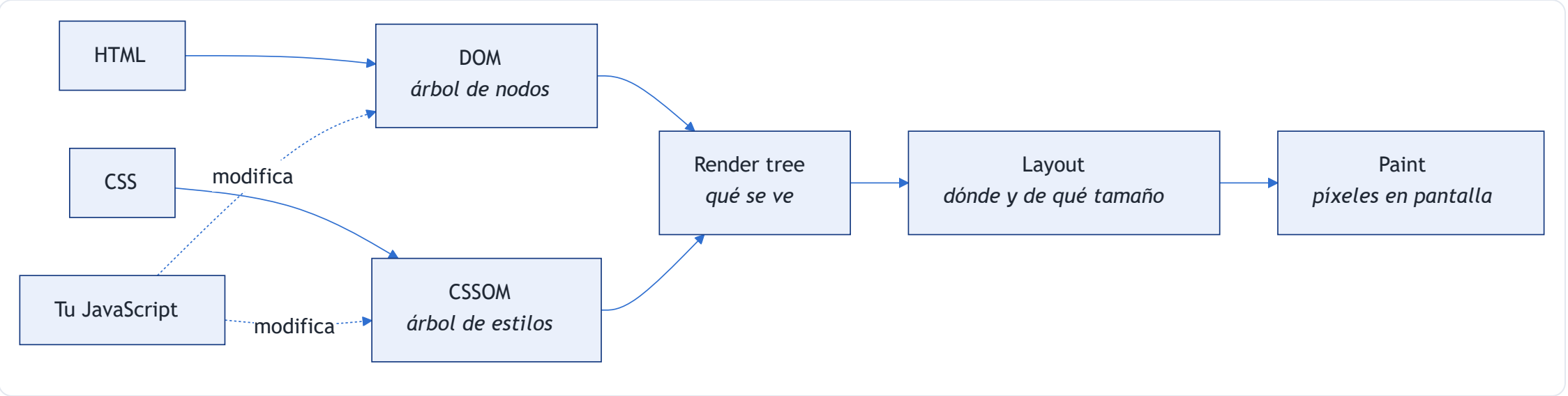
Antes de programar, entender el medio donde se ejecuta el código.

Modelo cliente-servidor



> Nuestro JavaScript viaja desde el servidor y se ejecuta **en el navegador del usuario**. No ve el disco ni la base de datos del servidor: pide datos por HTTP (Unidad 4).

Cómo el navegador construye la página



- Parseo → DOM y CSSOM → *render tree* → **layout** → **paint**.
- Cuando tu código toca el DOM, el navegador **rehace las etapas siguientes**: por eso se toca el DOM con mesura.

Las DevTools: tu instrumento de trabajo

Pestaña	Para qué sirve
Elements	Inspeccionar y editar en vivo el DOM y el CSS aplicado
Console	Ejecutar JavaScript y ver mensajes y errores
Network	Ver todas las peticiones HTTP: URL, estado, tiempo, respuesta
Sources	Ver el código y **depurar con breakpoints**
Application	Almacenamiento del navegador: `localStorage`, *cookies*…

> 💡 Aprende a usar los **breakpoints** de *Sources* en lugar de llenar el código de `console.log`: es la diferencia entre adivinar y **ver** qué hace tu programa.

Ahora tú · Explora las DevTools

⏱ **10 min** · 🔴 **sin IA** · ejercicio 12 de los apuntes

1. Abre cualquier web con **F12**. En **Elements**, cambia en vivo el texto de un titular.
2. En **Network**, recarga la página e identifica la petición del **documento HTML**: anota su **código de estado** y su **tiempo de carga**.
3. En **Console**, escribe `document.title` y luego `document.querySelectorAll("a").length`.

> Puesta en común: ¿qué código de estado tenía el documento? ¿Cuántas peticiones ha hecho la página en total?

3 · Cómo incluir JavaScript en una página

`<script>`, `defer`, `async`, `type="module"` y la consola.

La etiqueta `<script>`

```html
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <title>Mi página</title>
    <!-- defer: espera a que el HTML esté listo -
    <script src="app.js" defer></script>
  </head>
  <body>
    <h1 id="titulo">Hola</h1>
  </body>
</html>
```

Atributo	Cuándo se ejecuta	Uso
(ninguno)	Inmediatamente, **bloqueando** el HTML	Evitar
`defer`	Tras parsear todo el HTML, en orden	**Recomendado**
`async`	En cuanto se descarga, sin orden	Analítica, anuncios
`type="module"`	Como `defer`; habilita `import` / `export`	Proyectos modernos

> 💡 En proyectos nuevos, `type="module"`: módulos y *strict mode* automático.

DEMO EN VIVO

La consola del navegador

Tu mejor amiga para aprender y depurar. Desde el código puedes escribir en ella:

```
console.log("Hola, mundo");        // mensaje informativo
console.warn("Cuidado con esto"); // aviso
console.error("Algo ha fallado"); // error
console.table([1, 2, 3]);          // datos en forma de tabla
console.log({ nombre: "Ana", edad: 30 }); // objetos: se pueden desplegar
```

4 · Variables, tipos y coerción

`const` y `let`, siete tipos primitivos, y por qué `"5" + 1` es `"51"`.

Variables: `const`, `let` (y nunca `var`)

```
const PI = 3.14159; // no se puede reasignar
let contador = 0;   // sí se puede reasignar
contador = contador + 1;

const lista = [];
lista.push(1);      // ✅ válido: no reasignas
// lista = [2];     // ❌ TypeError
```

- `const`: no se puede **reasignar**. Úsalo por defecto.
- `let`: sí se puede reasignar. Solo cuando lo necesites.
- `var`: forma antigua, ámbito confuso. **Evítala**.

> ⚠️ `const` impide reasignar la variable, pero **no congela** el contenido de objetos o arrays: `lista.push(1)` es válido.

DEMO EN VIVO

Ámbito de bloque: por qué `var` confunde

```
if (true) {
  let x = 10;
  var y = 20;
}
// console.log(x); // ❌ ReferenceError: x no existe fuera del bloque
console.log("y (var):", y);    // ⚠️ 20: var "escapa" del bloque

// Quita el comentario de la línea de x y vuelve a ejecutar: verás el error real.
```

- `let` y `const` tienen **ámbito de bloque** (`{ ... }`): predecible.
- `var` tiene ámbito de **función** y se inicializa a `undefined` por el *hoisting*.
- `let` / `const` también se elevan, pero quedan en la **zona muerta temporal** hasta su declaración: errores claros en vez de `undefined` silenciosos.

Siete tipos primitivos

Tipo	Ejemplo
`string`	`"hola"`, `'a'`, `` `texto` ``
`number`	`42`, `3.14`, `-7`
`bigint`	`9007199254740993n`
`boolean`	`true`, `false`
`undefined`	declarada sin valor
`null`	ausencia **intencionada**
`symbol`	`Symbol("id")`

Todo lo demás es un **objeto** (arrays y funciones incluidos).

```
typeof "hola";        // "string"
typeof 42;            // "number"
typeof true;          // "boolean"
typeof undefined;     // "undefined"
typeof null;          // "object" ⚠️ rareza hist
typeof function(){}; // "function"
```

> JavaScript es de **tipado dinámico**: el tipo se decide en tiempo de ejecución. TypeScript (U2) añade tipos estáticos encima.

DEMO EN VIVO

Coerción de tipos: `==` frente a `===`

```
console.log('"5" + 3   →', "5" + 3);    // "53"  (number → string: concatena)
console.log('"5" - 3   →', "5" - 3);    // 2     (string → number)
console.log('1 == "1"  →', 1 == "1");   // true  (== convierte antes de comparar)
console.log('1 === "1" →', 1 === "1");  // false (=== compara valor Y tipo)

// Prueba a añadir estas joyas y explica cada una:
// console.log([] + {}, [] == false, null == undefined, null === undefined);
```

> 💡 Usa **siempre** `===` **y** `!==`. Evitan la coerción y previenen errores difíciles de detectar.

Truthy, falsy y conversión explícita

Los **únicos** valores *falsy*:

Valor	
`false`	el booleano
`0`, `-0`, `0n`	ceros
`""`	cadena vacía
`null`, `undefined`	ausencia
`NaN`	número inválido

Todo lo demás es truthy, incluidos `"0"`, `"false"`, `[]` y `{}`.

Mejor convertir **explícitamente**:

```
Number("42");       // 42
Number("hola");     // NaN
parseInt("42px");   // 42
String(42);         // "42"
Boolean(0);         // false
0.1 + 0.2 === 0.3; // false (IEEE 754)
```

> ⚠️ `input.value` **es siempre un string**, aunque el usuario escriba un número. Conviértelo con `Number()` o `"5" + 1` te dará `"51"`.

Ahora tú · 1.1 El detector de sorpresas

⏱ **15 min** · 🔴 **sin IA** · cuaderno 1, nivel 1

Antes de ejecutar nada, escribe en comentarios tu predicción para cada expresión. Después compruébalas en la consola y anota en cuáles fallaste y **por qué**.

```
"10" + 5;
"10" - 5;
10 == "10";
10 === "10";
true + true;
```

```
"" == false;
"" === false;
null ?? "vacío";
0 || "cero";
0 ?? "cero";
```

> Formato libre: `"10" + 5 → predije "105" → correcto: + con string concatena`.

5 · Operadores y cadenas de texto

Aritméticos, lógicos, ternario, y los modernos: `??`, `?.`, `...`

Operadores modernos imprescindibles

```
// Nullish coalescing (??):
// valor por defecto solo si es null o undefine
const usuarioNombre = null;
const nombre = usuarioNombre ?? "Anónimo";

// Optional chaining (?.):
// accede sin error si algo es null/undefined
const usuario = { direccion: null };
const ciudad = usuario?.direccion?.ciudad; // u
```

```
// Spread (...): expande elementos
const a = [1, 2];
const b = [...a, 3, 4]; // [1, 2, 3, 4]

// Rest (...): agrupa el resto
const [primero, ...resto] = [10, 20, 30];
// primero = 10, resto = [20, 30]

// Ternario
const edad = 20;
const msg = edad >= 18 ? "mayor" : "menor";
```

DEMO EN VIVO

`??` no es `||`

```
const descuento = 0;                  // 0 es un valor válido
console.log("con || :", descuento || 10);  // 10 ❌ (0 es falsy)
console.log("con ?? :", descuento ?? 10);  // 0  ✅ (0 no es null ni undefined)

const nombre = "";
console.log("con || :", nombre || "Anónimo"); // "Anónimo"
console.log("con ?? :", nombre ?? "Anónimo"); // ""  — ¿cuál querías?
```

> ⚠️ `0 || "x"` devuelve `"x"` porque `0` es *falsy*; `0 ?? "x"` devuelve `0`. Usa `??` cuando `0`, `""` o `false` sean valores **válidos**.

Strings y *template literals*

```
const simple = 'hola';
const doble = "mundo";

// Template literals: interpolación y multilíne
const nombre = "Ana";
const saludo = `Hola, ${nombre}. Tienes ${5 * 2
const multilinea = `Primera línea
Segunda línea`;
```

```
"Hola".length;                    // 4
"Hola".toUpperCase();             // "HOLA"
"  hola  ".trim();                // "hola"
"hola mundo".includes("mundo"); // true
"a,b,c".split(",");               // ["a","b","c"
"hola".replace("o", "0");         // "h0la" (solo
"coco".replaceAll("o", "0");      // "c0c0" (ES20
```

> Dentro de `${...}` cabe **cualquier expresión**: `${precio * 1.1}`, `${evento.toUpperCase()}`.

Ahora tú · 1.2 La máquina de entradas y 1.3 El precio justo

⏱ **2 × 10 min** · 🔴 **sin IA** · cuaderno 1, nivel 1

1.2 `crearEntrada(nombre, evento, precio = 15)` devuelve, con un **template literal multilínea**, una entrada formateada con el precio + 10 % de gestión **calculado dentro de la interpolación**.

```
============================
  🎟  JS FEST 2026
  Asistente: Ada Lovelace
  Precio final: 16.5 €
============================
```

1.3 `precioFinal(precio, descuento)`:

- `descuento` `null` / `undefined` → 0 % (con `??`, **no** `||`).
- Un descuento de `0` es válido.
- Precio negativo → `"Precio no válido"`.

```
precioFinal(100, 25); // 75
precioFinal(100, 0);  // 100
precioFinal(100);     // 100
precioFinal(-5, 10);  // "Precio no válido"
```

6 · Control de flujo y funciones

Condicionales, bucles, *arrow functions*, parámetros por defecto y *closures*.

Condicionales y bucles

```
const nota = 7;
if (nota >= 9) {
  console.log("Sobresaliente");
} else if (nota >= 5) {
  console.log("Aprobado");
} else {
  console.log("Suspenso");
}

switch (dia) {
  case "sábado":
  case "domingo":
    console.log("Fin de semana"); break;
  default:
    console.log("Laborable");
}
```

```
for (let i = 0; i < 3; i++) { /* 0,1,2 */ }

// for...of: recorre VALORES
for (const fruta of ["manzana", "pera"]) {
  console.log(fruta);
}

// for...in: recorre CLAVES de un objeto
const persona = { nombre: "Ana", edad: 30 };
for (const clave in persona) {
  console.log(clave, persona[clave]);
}

let n = 3;
while (n > 0) { n--; }
```

> 💡 `for...of` para valores, `for...in` solo para claves. Para transformar datos: `map`, `filter`, `reduce` (U2).

Ahora tú · 1.4 Cuenta atrás de lanzamiento

⏱ **15 min** · 🔴 **sin IA** · cuaderno 1, nivel 1

Cuenta atrás del **10 al 0** con un bucle descendente. Reglas de la sala de control:

- Número **par**: `n... (comprobando sistemas)`.
- Número **impar**: solo `n...`.
- En **3, 2 y 1** se añade `¡Abróchense!`.
- En **0**: `🚀 ¡Despegue!` y el bucle termina.

```
10... (comprobando sistemas)
9...
...
3... ¡Abróchense!
2... (comprobando sistemas) ¡Abróchense!
1... ¡Abróchense!
🚀 ¡Despegue!
```

> Pista: construye la línea en una variable `let linea = \`${n}...\`;` y ve concatenando condiciones. `Par:` `n % 2 === 0`.

Funciones: tres formas de escribirlas

```
// Declaración
function sumar(a, b) {
  return a + b;
}

// Expresión (guardada en una variable)
const restar = function (a, b) {
  return a - b;
};
```

```
// Arrow functions: la forma moderna y compacta
const sumar = (a, b) => a + b;       // return impl
const cuadrado = x => x * x;         // un parámetro
const saludar = () => console.log("hola");

// Varias líneas: llaves y return explícito
const dividir = (a, b) => {
  if (b === 0) return null;
  return a / b;
};
```

> Las *arrow functions* dominan en *callbacks* y métodos de array (U2). Diferencias con `this`: U2.

DEMO EN VIVO

Parámetros por defecto, *rest* y *closures*

```
function saludar(nombre = "invitado") {
  return `Hola, ${nombre}`;
}
saludar();       // "Hola, invitado"
saludar("Ana");  // "Hola, Ana"

// Rest: número variable de argumentos
function sumarTodos(...numeros) {
  return numeros.reduce((t, n) => t + n, 0);
}
sumarTodos(1, 2, 3, 4); // 10
```

```
function crearContador() {
  let cuenta = 0;             // variable "privada"
  return () => ++cuenta;      // recuerda 'cuenta'
}

const a = crearContador();
const b = crearContador();
console.log(a(), a(), a()); // 1 2 3
console.log(b());           // 1 → estado independiente
```

> Un **closure** es una función que recuerda el ámbito en el que fue creada. Nadie fuera puede tocar `cuenta`.

Arrays y objetos: lo mínimo para seguir

```
// Array: colección ordenada, índice desde 0
const numeros = [10, 20, 30];
numeros[0];         // 10
numeros.length;     // 3
numeros.push(40);   // [10, 20, 30, 40]
```

```
// Objeto: pares clave-valor
const persona = {
  nombre: "Ana",
  edad: 30,
  saludar() {
    return `Hola, soy ${this.nombre}`;
  },
};
persona.nombre;      // "Ana"
persona["edad"];     // 30
persona.saludar();   // "Hola, soy Ana"
```

> Se estudian a fondo en la **Unidad 2** (métodos de array, desestructuración, referencia frente a valor).

Misión 1 · El oráculo de los números 🔮

🟡 **IA como pair · ⏱ 60–90 min** · cuaderno 1, nivel 2

- 🎯 **Objetivo:** el juego «Adivina el número» (1–100) completo en el navegador.
- 🧰 **Practicas:** `const` / `let`, `Number()` y `===`, condicionales, *template literals*, `querySelector`, `textContent`, evento `click`.

```
// app.js — el oráculo elige su número secreto:
const secreto = Math.floor(Math.random() * 100) +
console.log("(psst... el secreto es", secreto, ")
```

Fases (checkpoints)

1. **Conexión:** `app.js` carga con `defer`; selecciona input, botón y párrafos.
2. **Primera consulta:** `click` → lee el valor, `Number()`, muéstralo.
3. **El oráculo responde:** mayor / menor / correcto; valor vacío o fuera de 1–100 → aviso sin gastar intento.
4. **El marcador:** `let intentos` visible en pantalla.
5. **Fin de partida:** al acertar, `boton.disabled = true`.

Misión 1 · Criterios de aceptación y retos

✅ No está terminada hasta marcar todo:

- ☐ El script se carga con `defer` y no hay errores en consola.
- ☐ Vacío o fuera de rango: aviso y **no** incrementa intentos.
- ☐ Pistas mayor/menor correctas en todos los casos.
- ☐ El contador se actualiza en cada consulta válida.
- ☐ Al acertar: mensaje con los intentos y botón desactivado.
- ☐ Ningún `console.log` con el secreto en la versión final.

🚀 Retos extra

- Límite de **7 intentos**: el oráculo revela el número y bloquea el botón (logro «Francotirador»).
- Botón **«Nueva profecía»** que reinicia la partida.
- Historial de intentos en un array: `Has probado: 50, 75, 62`.

> Semáforo 🟡: pídele a la IA críticas y pistas («¿por qué falla esto?»), no la solución entera. El código lo escribes tú.

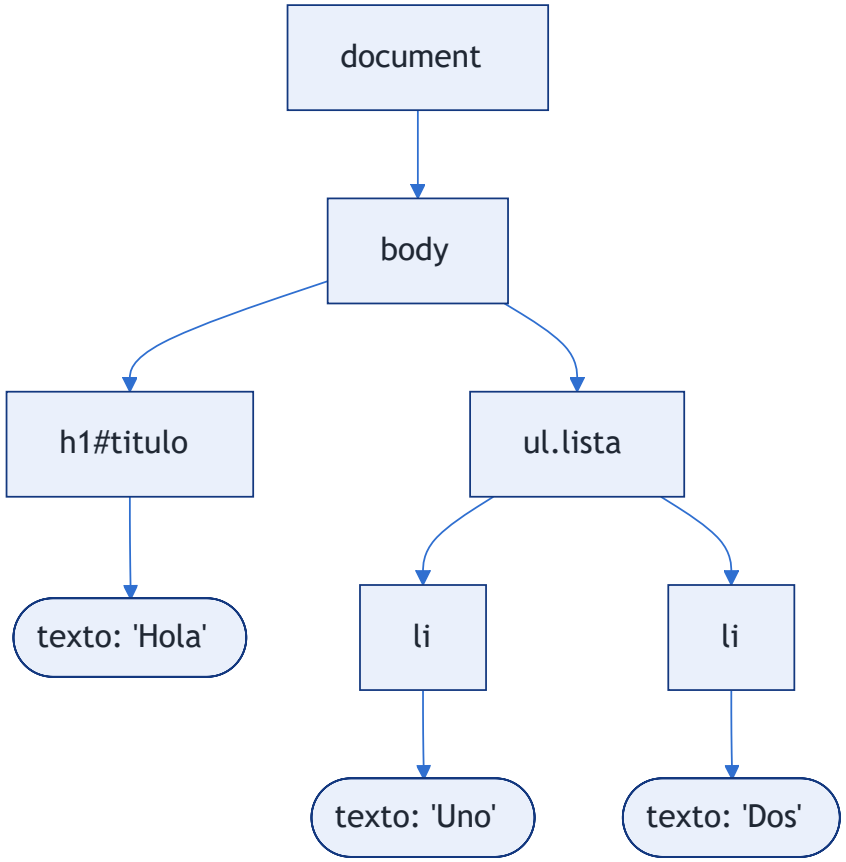
7 · El DOM: manipular la página

Seleccionar, modificar, responder a eventos y **delegar**.

El DOM es un árbol

```
<body>
  <h1 id="titulo">Hola</h1>
  <ul class="lista">
    <li>Uno</li>
    <li>Dos</li>
  </ul>
</body>
```

- El **texto también es un nodo**: por eso `textContent` es una propiedad.
- Todo elemento tiene **padre** y **hermanos**: eso aprovecha la delegación.



Seleccionar y modificar

```
// El más versátil: selectores CSS
const titulo = document.querySelector("#titulo"
const botones = document.querySelectorAll(".bot
const primero = document.querySelector("ul li")
```

💡 Busca los elementos **una vez**, al inicio, y guárdalos en constantes: repetir `querySelector` en cada clic es trabajo inútil.

```
titulo.textContent = "Nuevo título";  // texto
titulo.innerHTML = "<em>Hola</em>";   // HTML i
titulo.style.color = "crimson";       // estilo
titulo.classList.add("destacado");    // añade
titulo.classList.toggle("oculto");    // añade/
titulo.setAttribute("data-id", "42"); // atribu
```

Desde JS se cambian **clases**; los estilos siguen viviendo en el CSS.

DEMO EN VIVO

`textContent` o `innerHTML`: la puerta del XSS

Escribe `<img src=x onerror="alert('XSS')">` en la caja y compara.

```
<!doctype html>
<html lang="es">
<body style="font-family: system-ui; padding: 8px">
  <input id="entrada" size="42" value="&lt;b&gt;hola&lt;/b&gt;">
  <button id="pintar">Insertar</button>
  <p>Con <code>textContent</code>: <span id="seguro"></span></p>
  <p>Con <code>innerHTML</code>: <span id="peligroso"></span></p>
  <script>
    document.getElementById("pintar").addEventListener("click", () => {
      const texto = document.getElementById("entrada").value;
      document.getElementById("seguro").textContent = texto;   // TEXTO
      document.getElementById("peligroso").innerHTML = texto;  // se INTERPRETA
    });
  </script>
</body>
</html>
```

DEMO EN VIVO

Eventos: responder a la interacción

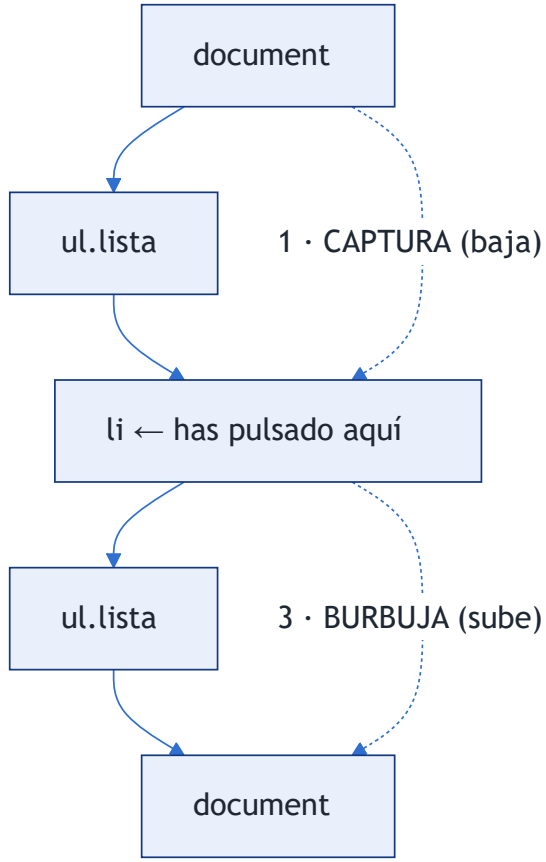
```
const boton = document.querySelector("#miBoton");
boton.addEventListener("click", () => {
  console.log("¡Me han pulsado!");
});

// El objeto 'event' describe el suceso
const input = document.querySelector("#nombre");
input.addEventListener("input", (event) => {
  console.log("Valor:", event.target.value);
});
```

Habituales: `click`, `input`, `change`, `submit`, `keydown`, `mouseover`, `load`, `DOMContentLoaded`.

```
<!doctype html>
<html lang="es">
<body style="font-family: system-ui; padding: 8px
  <button id="incrementar">Suma uno</button>
  <p>Llevas <strong id="valor">0</strong> clics.</
  <script>
    const boton = document.querySelector("#increm
    const salida = document.querySelector("#valor
    let cuenta = 0;
    boton.addEventListener("click", () => {
      cuenta++;
      salida.textContent = cuenta;
    });
  </script>
</body>
</html>
```

Propagación: el evento viaja por el árbol



- Primero **baja** desde `document` hasta el elemento pulsado (*captura*).
- Llega al **objetivo** (`target`).
- Después **sube** de vuelta (*burbuja*): la fase que se usa casi siempre.

Que el evento **suba** tiene una consecuencia práctica: un solo listener en el **padre** se entera de lo que pasa en todos los hijos.

Eso es la **delegación de eventos**.

DEMO EN VIVO

Delegación: un listener en vez de doscientos

```
<!doctype html>
<html lang="es">
<body style="font-family: system-ui; padding: 8px">
  <ul id="lista"><li>Manzana</li><li>Plátano</li><li>Cereza</li></ul>
  <button id="añadir">Añadir fruta</button>
  <p id="aviso"></p>
  <script>
    const lista = document.getElementById("lista");
    const aviso = document.getElementById("aviso");
    // UN solo listener para todos los <li>, presentes y futuros
    lista.addEventListener("click", (event) => {
      const li = event.target.closest("li");
      if (!li || !lista.contains(li)) return;   // clic fuera de un <li>
      li.style.textDecoration = li.style.textDecoration ? "" : "line-through";
      aviso.textContent = `Has pulsado: ${li.textContent}`;
    });
```

Misión 2 · La forja de claves 🔐

🟡 **IA como pair** · ⏱ **60–90 min** · cuaderno 1, nivel 2

- 🎯 **Objetivo:** generador de contraseñas con longitud y materiales configurables (letras, cifras, símbolos).
- 🧰 **Practicas:** funciones con parámetros, bucles, strings e indexación, booleanos, *checkboxes*, `classList`.

Fases

1. **La receta:** `forjarClave(longitud, usarNumeros, usarSimbolos)` → string. Función **pura**: no toca el DOM.
2. **El alfabeto:** letras + `"0123456789"` + `"!@#$%&*?"` según los booleanos.
3. **El martillo aleatorio:** `Math.floor(Math.random() * alfabeto.length)`.
4. **La entrega:** botón → `Number()`, `.checked`, mostrar en `#clave`.
5. **Control de calidad:** longitud fuera de 4–32 → aviso.

✅ Criterios

- ☐ `forjarClave` es pura y devuelve siempre un string.
- ☐ La clave tiene exactamente la longitud pedida.
- ☐ Sin casillas marcadas, solo letras.
- ☐ Cada pulsación genera una clave distinta.
- ☐ Longitud fuera de rango: aviso y sin clave.

🚀 **Retos:** medidor de temple (`débil` / `aceptable` / `legendaria` con `classList`), garantizar un carácter de cada grupo, historial de 3 claves.

Misión 3 · La tarjeta holográfica 🪪

🟡 **IA como pair** · ⏱️ **60–90 min** · cuaderno 1, nivel 2

- 🎯 **Objetivo:** tarjeta de presentación que reacciona a eventos, cambia de tema y esconde un secreto.
- 🧰 **Practicas:** `mouseover` / `mouseout` / `input` / `keydown`, `classList`, arrays y un **closure de verdad**.

Fases

1. **Holografía:** `mouseover` añade `brillo`, `mouseout` la quita.
2. **Edición en vivo:** `input` actualiza `#nombre`; vacío → «Tu nombre».
3. **El alternador (closure):** `crearAlternador(temas)` devuelve una función que da el siguiente tema y vuelve al principio (`%`). El índice es **privado**.
4. **Conectar el botón:** quitar la clase del tema actual, poner la nueva.
5. **El secreto:** `keydown` en `document`, tecla `h` → lema «✨ Modo holograma desbloqueado».

✅ Criterios

- ☐ Brilla al entrar el ratón y deja de brillar al salir.
- ☐ El nombre se actualiza tecla a tecla; vacío → texto por defecto.
- ☐ `crearAlternador` usa un closure: el índice no es global.
- ☐ Dos alternadores con arrays distintos son independientes.
- ☐ El ciclo de temas es infinito.
- ☐ La tecla `h` desbloquea el secreto.

🚀 **Retos:** contador de visitas holográficas, `ArrowRight` reutiliza el alternador, `dblclick` alterna el lema.

8 · Una primera mirada a la asincronía

JavaScript tiene **un solo hilo**: encarga la tarea y sigue; cuando termina, ejecuta un *callback*.

DEMO EN VIVO

`setTimeout` y el orden de ejecución

```
console.log("1");

setTimeout(() => {
  console.log("3"); // se ejecuta DESPUÉS, aunque pongamos 0 ms
}, 0);

console.log("2");

// Salida: 1, 2, 3
```

- El navegador no se «congela» esperando: **encola** la tarea y sigue con lo siguiente.
- Es el mecanismo del reto jefe: un `setTimeout` cuya función vuelve a programar el siguiente.

> 📌 En la **Unidad 2**: *event loop*, Promesas y `async/await` (por ejemplo, para pedir datos con `fetch`).

Buenas prácticas y errores comunes

- ✅ `const` por defecto, `let` solo si reasignas. Olvídate de `var`.
- ✅ Compara siempre con `===` / `!==`.
- ✅ Nombres descriptivos en `camelCase` (`numeroDeUsuarios`, no `n`).
- ✅ `console.log` para inspeccionar, pero retíralo del código final.
- ✅ *Strict mode* (automático con `type="module"`).
- ⚠️ No confundas `=` (asignación) con `===` (comparación).
- ⚠️ Los índices de array empiezan en `0`.
- ⚠️ Evita variables globales: encierra el código en módulos o funciones.
- ⚠️ `input.value` es un string; `innerHTML` interpreta HTML.

Reto jefe · Caza al bug 👾

🟢 **IA libre (declarada)** · ⏱️ **2–3 h** · cuaderno 1, nivel 3

Un *whack-a-mole* en vanilla JS: los bugs aparecen y desaparecen en una cuadrícula de 3 × 3 y hay que aplastarlos antes de que acabe el tiempo.

Requisitos mínimos

1. Cuadrícula de **9 casillas** (nueve `<button>`) y marcador con **puntos** y **tiempo**.
2. «Jugar»: cada cierto intervalo un 🐛 aparece en una casilla **aleatoria** y desaparece. `setTimeout` **encadenado**.
3. Clic en bug: +1 y desaparece. Clic en vacía: −1 (mínimo 0).
4. La partida dura **30 s**; al terminar, mensaje según puntuación y **ningún bug más aparece**.
5. Funciones con nombre claro (`aparecerBug`, `finDePartida`, `actualizarMarcador`…), cero `var`, cero errores en consola.

```
<main>
  <h1>👾 Caza al bug</h1>
  <p>Puntos: <span id="puntos">0</span> ·
     Tiempo: <span id="tiempo">30</span> s</p>
  <button id="jugar">▶️ Jugar</button>
  <section id="tablero">
    <!-- 9 botones .casilla -->
  </section>
</main>
```

🚀 **Retos:** dificultad creciente cada 10 s · bug dorado ✨ (5 puntos) · récord de la sesión.

Reto jefe · Rúbrica de autoevaluación

Criterio	Básico	Bien	Excelente
Funcionalidad	El bug aparece y se puntúa, con fallos ocasionales	Puntos, reloj y fin de partida funcionan siempre	+ al menos un reto extra funcionando
DOM y eventos	Mezcla `innerHTML` y estilos sin criterio	`textContent`, `classList` y `addEventListener` correctos	Manejadores con `for...of` o delegación; `event.target` con soltura
Calidad del código	Globales sueltas y nombres crípticos	`const` / `let` bien elegidos, funciones pequeñas y claras	Estado agrupado, funciones puras, cero repetición
Robustez	Se rompe (doble clic en «Jugar», clics tras el final)	Los casos raros están controlados	Ningún `setTimeout` sobrevive al fin; consola impecable
Comprensión	Sabes qué hace a grandes rasgos	Explicas cada función a un compañero	Justificas cada decisión (¿por qué una clase y no una variable?)

> La fila **Comprensión** es la que se examina en la defensa oral de la Arena.

Hacia la Arena · Misión M1 «El Despertar del DOM»

Sin frameworks. Sin librerías. Tú contra el navegador.

Qué se entrega: una URL de GitHub (repo o subcarpeta) con una página interactiva en HTML + CSS + JS puro: un mini-juego, un generador, un tablero… Prohibido React, prohibido jQuery.

Cómo se puntúa (/100)

- **30** checks automáticos: README, repo limpio, ≥ 5 commits, tecnología correcta, estructura.
- **70** rúbrica evaluada por IA: DOM **20** · Eventos **15** · Fundamentos JS **15** · Calidad **10** · Originalidad **10**.
- 🎁 Bonus: modo oscuro con una **tecla secreta**.

La nota es provisional hasta la defensa oral. La IA genera 3 preguntas sobre **tu** código; en el ranking cuenta el mejor intento.

- README con sección **«Uso de IA»**: qué pediste, qué cambiaste, qué entiendes.
- Sin *handlers* inline (`onclick="…"`), sin `var`, sin `innerHTML` con datos del usuario.
- HTML, CSS y JS en archivos propios.

> Ejemplo de entrega: `demo/R1/` (Caza el Bug del profesor).

Entrena la defensa oral

Preguntas del mismo corte que las de la Arena. Se responden en 1–2 minutos **si has entendido**; de memoria son imposibles.

1. Tu página crea elementos nuevos según juega el usuario. ¿Un listener en cada uno o uno solo en el contenedor? ¿Cómo sabe tu código **en qué elemento** se ha pulsado?
2. Enséñame una línea donde pintes texto. ¿`textContent` o `innerHTML`? Si ese texto lo escribiera el usuario, ¿qué podría pasar?
3. ¿Por qué seleccionas los elementos al arrancar y no dentro de cada *handler*? ¿Qué pasaría **sin** `defer`?
4. ¿Dónde vive el estado de tu aplicación: en variables o en el DOM? Defiende tu elección.
5. Tienes un `const` con un array al que haces `push`. ¿No debería fallar?
6. Comparas el valor de un input con un número: ¿`==` o `===`? ¿Qué conversión ocurre?

> Las ocho preguntas con respuesta modelo están en la **autocomprobación** de la unidad, en la Arena (`/teoria`).

Tabla de logros del cuaderno 1

Logro 🏅	Cómo desbloquearlo
🔥 **Manos calientes**	Completar los 4 ejercicios del Nivel 1
🔮 **Vidente verificado** · ⚒️ **Herrero de claves** · 🪪 **Identidad holográfica**	Terminar cada misión con todos los criterios marcados
👾 **Cazador de bugs**	Superar el reto jefe con al menos «Bien» en toda la rúbrica
🎯 **Francotirador**	Acertar el número del oráculo en 7 intentos o menos
🗡️ **Clave legendaria**	Implementar el medidor de temple de la Misión 2
✨ **Guardián del secreto**	El atajo oculto de la Misión 3 y el bug dorado del jefe
🧠 **Sin red**	Resolver una misión completa sin abrir ninguna pista
🐛 **Depurador nato**	Resolver un fallo propio con un *breakpoint* en *Sources*
💯 **Pleno al quince**	«Excelente» en los cinco criterios de la rúbrica del jefe

Recursos y próxima unidad

Para profundizar

- MDN — JavaScript (en español): https://developer.mozilla.org/es/docs/Web/JavaScript
- MDN — Primeros pasos en JavaScript: https://developer.mozilla.org/es/docs/Learn_web_development/Core/Scripting
- *Eloquent JavaScript* (libro gratuito): https://eloquentjavascript.net/
- javascript.info — The Modern JavaScript Tutorial: https://javascript.info/
- Especificación ECMA-262: https://ecma-international.org/publications-and-standards/standards/ecma-262/

Unidad 2 — JavaScript avanzado y herramientas del frontend

- Arrays y objetos a fondo: `map`, `filter`, `reduce`, desestructuración, inmutabilidad.
- `this`, prototipos y clases; `Map` y `Set`.
- El *event loop*, promesas, `async/await` y `fetch`.
- Módulos ES, npm, **Vite** y una introducción a **TypeScript**.
- Misión M2 «Async Odyssey».